Linear Regression : The simplest model that learns

Building the intuition and the mathematics, one idea at a time

Part 1 of the series *Neural Networks learning Guide*

Note for reading this document

Each section adds exactly one idea, and each idea arrives in three forms: a picture or an everyday intuition, the equation that captures it, and a small calculation you can check by hand. We use the same five data points from start to finish, so every number you see is one you can reproduce with a pencil.

By the end you will have derived the best-fit line two different ways (with calculus, and by gradient descent), met the matrix form that scales to many inputs, understood why "least squares" is a natural choice, and seen that a linear regression model is already a one-neuron neural network.

1. The problem: learning a rule from examples

Suppose we have collected examples, each consisting of something we can measure (the **input**, written x) and something we would like to predict (the **target**, written y). A dataset of n examples is a list of pairs $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$. The subscript i labels the example, so x_i is the input of the i-th example.

The goal is a rule that takes a *new* input and returns a good guess for its target. Writing the rule as a function f, the guess is $\hat{y} = f(x)$. The little hat on $\hat{y}$ always means "predicted"; the plain y is what really happened.

Every machine-learning method, from the humble line in this document to a network with billions of parameters, makes three decisions:

- **The model:** what family of rules f are we willing to consider?
- **The loss:** how do we score a rule, so that "better" has a number attached?
- **The optimiser:** how do we search the family for the rule with the best score?

2. A tiny dataset to think with

Five students recorded the hours they studied (x) and their mark on a quiz out of 10 (y). The numbers are illustrative, chosen so the arithmetic stays friendly.

Student i	Hours studied x_i	Mark y_i
1	1	2.2
2	2	2.8
3	3	4.5
4	4	3.7
5	5	5.5

More study tends to go with higher marks, but not perfectly: student 4 studied longer than student 3 and scored less. Real data always has this scatter, and any honest model must cope with it.

Two summary numbers will keep reappearing. The **mean** of a list is its balance point:

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i = \frac{1+2+3+4+5}{5} = 3, \quad \bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i = \frac{18.7}{5} = 3.74$$

Here $\sum_{i=1}^{n}$ (the capital Greek sigma) is shorthand for "add up the terms for $i = 1$ up to $i = n$".

3. Step one: the model is a straight line

Begin with the humblest predictor imaginable: ignore x and always guess the same number b. The best such guess turns out to be the mean $\bar{y}$, which makes sense, since it is the balance point of the targets.

That ignores useful information. Study hours *do* seem to matter, so let the prediction move with x. The simplest way to do that is to let it rise or fall at a constant rate:

$$\hat{y} = w\,x + b$$

The two numbers w and b are the **parameters** of the model, the dials that learning will set:

- The **weight** (or slope) w says how much the prediction changes when x grows by one unit. If $w = 0.75$, one extra hour of study adds 0.75 marks to the prediction.
- The **bias** (or intercept) b is the prediction when $x = 0$. It slides the whole line up or down without changing its tilt.

Every pair (w, b) is a different line, so the model is really a *family* of candidate rules, and learning means choosing one member of the family. For the i-th example the model predicts $\hat{y}_i = w\,x_i + b$.

More than one input

Real problems have several inputs: hours studied, hours slept, number of practice papers, and so on. Give each input its own weight and add them up:

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_d x_d + b = \sum_{j=1}^{d} w_j x_j + b$$

This sum of "weight times input" plus a bias is called a **dot product** plus a bias, and it is the exact computation performed by a single artificial neuron. We return to that in Section 11.

4. Step two: measuring how wrong a line is

To compare candidate lines we need a score. Start with the most natural quantity, the gap between what happened and what the line predicted. This gap is the **residual**:

$$e_i = y_i - \hat{y}_i$$

A positive residual means the line guessed too low; a negative one means too high. Graphically, residuals are the vertical segments between each point and the line (the pink segments in Figure 1).

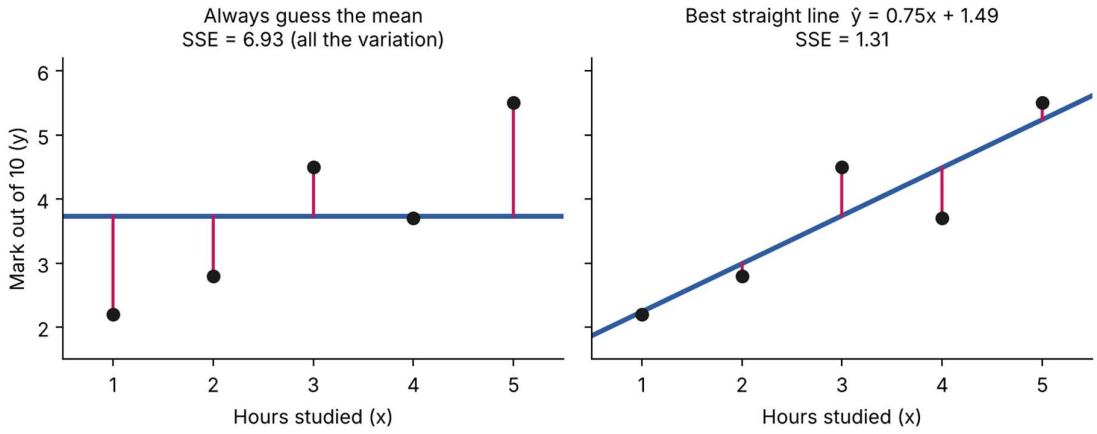


Figure 1. Left: ignoring x and always guessing the mean leaves large gaps. Right: the best straight line shrinks them. The pink segments are the residuals.

How should we combine the five residuals into one score? Simply adding them fails, because positive and negative errors cancel. A terrible line that overshoots one student by 3 marks and undershoots another by 3 would score a perfect zero. We need each error to count as a positive amount. Two classic choices are $|e_i|$ (the absolute error) and e_i^2 (the squared error). Least squares uses the square, for four reasons:

- It is positive, so errors cannot cancel.
- It punishes a big miss far more than a small one (an error of 2 costs four times an error of 1), which matches the feeling that large mistakes are worse.
- It is smooth everywhere, so calculus works cleanly. The absolute value has a kink at zero.
- It corresponds to a natural assumption about noise, shown in Section 9.

Averaging the squared residuals over all examples gives the **mean squared error**, our **loss function**:

$$L(w, b) = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 = \frac{1}{n}\sum_{i=1}^{n}(y_i - w\,x_i - b)^2$$

Notice what this equation does: it takes any choice of the two dials (w, b) and returns a single number saying how badly that line fits. Learning has become an optimisation problem: **find the** w and b that make L as small as possible.

Three candidate lines on our data show how the loss ranks them:

Candidate line	w	b	Loss L
Always guess 0	0	0	15.374
Always guess the mean	0	3.74	1.386
A tilted line (found in Section 5)	0.75	1.49	0.261

The loss as a landscape

Picture a map in which the east-west direction is w and the north-south direction is b. Above each location, imagine the height $L(w, b)$. Because L is built from squares, this surface is a smooth **bowl** with a single lowest point. The best line sits at the bottom of the bowl. Figure 2 (left) shows the bowl from above as contour lines, like a hiking map.

5. Step three: finding the bottom with calculus

At the bottom of a bowl the ground is level in every direction: the slope is zero. For a function of two variables, "slope in each direction" means a **partial derivative**, written with the curly ∂ and meaning "how fast does L change if I nudge only this variable?". The bottom is where both partial derivatives vanish.

Solving for the bias

Differentiate L with respect to b using the chain rule (the derivative of u^2 is $2u$, and $u = y_i - wx_i - b$ has derivative -1 with respect to b):

$$\frac{\partial L}{\partial b} = -\frac{2}{n}\sum_{i=1}^{n}(y_i - w\,x_i - b) = 0$$

Multiply by $-n/2$ and split the sum: $\sum y_i - w\sum x_i - n\,b = 0$. Dividing by n gives

$$b = \bar{y} - w\,\bar{x}$$

This says the best line always passes through the point of means $(\bar{x}, \bar{y})$, the centre of mass of the data.

Solving for the weight

Now differentiate with respect to w. The inner derivative is $-x_i$:

$$\frac{\partial L}{\partial w} = -\frac{2}{n}\sum_{i=1}^{n} x_i(y_i - w\,x_i - b) = 0$$

Substitute $b = \bar{y} - w\bar{x}$, so that $y_i - wx_i - b = (y_i - \bar{y}) - w(x_i - \bar{x})$, and rearrange:

$$\sum_{i=1}^{n} x_i(y_i - \bar{y}) = w\sum_{i=1}^{n} x_i(x_i - \bar{x})$$

Two small facts tidy this up. Deviations from a mean always add to zero, so $\sum \bar{x}\,(y_i - \bar{y}) = 0$ and $\sum \bar{x}\,(x_i - \bar{x}) = 0$. We may therefore replace x_i by $x_i - \bar{x}$ on both sides, giving the famous result:

$$w = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2} = \frac{S_{xy}}{S_{xx}}$$

The numerator S_{xy} measures how much x and y move together (positive if they rise together, negative if one rises as the other falls). The denominator S_{xx} measures how spread out x is. The slope is "co-movement per unit of spread". Equivalently, $w = r\, s_y/s_x$, where r is the correlation coefficient and s_x, s_y are the standard deviations.

The calculation on our data

i	x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$	$(x_i - \bar{x})^2$
1	1	2.2	−2	−1.54	3.08	4
2	2	2.8	−1	−0.94	0.94	1
3	3	4.5	0	0.76	0	0
4	4	3.7	1	−0.04	−0.04	1
5	5	5.5	2	1.76	3.52	4
Sum					**7.50**	**10**

$$w = \frac{7.5}{10} = 0.75, \quad b = 3.74 - 0.75 \times 3 = 1.49$$

The fitted line is $\hat{y} = 0.75\,x + 1.49$: each extra hour of study is associated with 0.75 more marks. Here are its predictions and residuals:

x_i	y_i	$\hat{y}_i$	$e_i = y_i - \hat{y}_i$
1	2.2	2.24	−0.04
2	2.8	2.99	−0.19
3	4.5	3.74	0.76
4	3.7	4.49	−0.79
5	5.5	5.24	0.26
Sum			**0.00**

6. How good is the line? Total, explained and unexplained variation

A loss of 0.261 is small, but small compared with what? We need a yardstick. The natural baseline is the "always guess the mean" model, whose squared error is the **total variation** in the targets:

$$SST = \sum_{i=1}^{n} (y_i - \bar{y})^2 = 6.932$$

The line's own leftover error is the **sum of squared errors**:

$$SSE = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2 = 1.307$$

The fraction of the baseline's error that the line removes is the **coefficient of determination**:

$$R^2 = 1 - \frac{SSE}{SST} = 1 - \frac{1.307}{6.932} \approx 0.811$$

So about 81% of the variation in marks is accounted for by study hours. A value of 1 would mean a perfect fit; 0 means no better than guessing the mean. Another useful summary is the **root mean squared error**, $\sqrt{L} = \sqrt{0.2614} \approx 0.51$ marks, which is in the same units as y: our typical prediction is off by about half a mark.

R^2 describes fit, not truth. With only five points it is unreliable, it says nothing about whether study causes higher marks, and a high value can hide a curved relationship. Always look at the residuals.

7. Learning by walking downhill: gradient descent

The formulas above solved the problem in one shot. But a formula like that exists for very few models. For neural networks there is none, so we need a general-purpose method that works for *any* model whose loss we can differentiate. That method is **gradient descent**, and it is the engine of modern machine learning.

The intuition. Imagine standing on the bowl's slope in thick fog. You cannot see the bottom, but you can feel which way the ground tilts under your feet. Take a small step in the steepest downhill direction, feel the slope again, and repeat. You will eventually reach the bottom.

The slope information is exactly what we already computed. The two partial derivatives, collected together, form the **gradient**, which points uphill. Written with the residuals e_i:

$$\frac{\partial L}{\partial w} = -\frac{2}{n}\sum_{i=1}^{n} x_i\, e_i\,, \quad \frac{\partial L}{\partial b} = -\frac{2}{n}\sum_{i=1}^{n} e_i$$

Stepping *against* the gradient gives the **update rule**:

$$w \leftarrow w - \eta\,\frac{\partial L}{\partial w}, \quad b \leftarrow b - \eta\,\frac{\partial L}{\partial b}$$

The arrow means “replace the old value with the new one”. The small positive number η (Greek eta) is the **learning rate**, the size of each step.

One step by hand

Start from the poorest possible guess, $w = 0$ and $b = 0$, with $\eta = 0.01$. Then every prediction is 0, so $e_i = y_i$. We need $\sum x_i y_i = 2.2 + 5.6 + 13.5 + 14.8 + 27.5 = 63.6$ and $\sum y_i = 18.7$:

$$\frac{\partial L}{\partial w} = -\frac{2}{5}(63.6) = -25.44, \quad \frac{\partial L}{\partial b} = -\frac{2}{5}(18.7) = -7.48$$

$$w \leftarrow 0 - 0.01(-25.44) = 0.2544, \quad b \leftarrow 0 - 0.01(-7.48) = 0.0748$$

Both gradients were negative (increasing w and b lowers the loss), so both parameters moved up, and the loss fell from 15.374 to 9.174. Repeating the step gives:

Steps taken	w	b	**Loss** L
0	0.000	0.000	15.374
1	0.254	0.075	9.174
10	0.996	0.319	0.570
100	0.985	0.641	0.393
1,000	0.761	1.450	0.2617
5,000	0.750	1.490	0.2614

The answer converges to the same $w = 0.75$ and $b = 1.49$ that calculus gave us, which is the reassuring part. The path (Figure 2, left) shows something instructive: a fast plunge into a long, narrow valley, then a slow crawl along its floor.

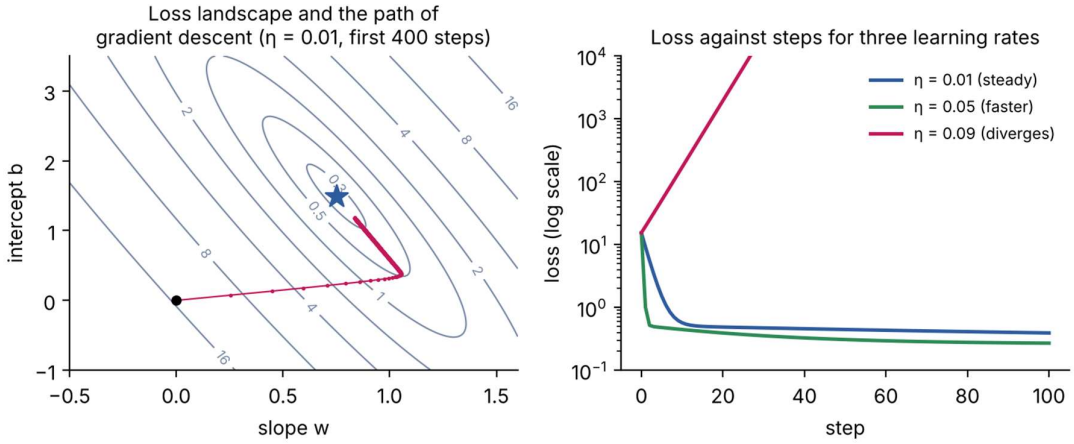


Figure 2. Left: contours of the loss with the gradient-descent path (pink) heading for the optimum (star). Right: loss against steps for three learning rates.

Choosing the learning rate

To see why η matters, look at the simplest possible bowl, $L(w) = a\,(w - w^*)^2$ with $a > 0$, whose bottom is at w^*. Its derivative is $2a(w - w^*)$, so one update gives

$$w_{new} - w^* = (w - w^*) - 2a\eta\,(w - w^*) = (1 - 2a\eta)\,(w - w^*)$$

Each step multiplies the remaining distance by $(1 - 2a\eta)$. The distance shrinks only if $|1 - 2a\eta| < 1$, that is, if $0 < \eta < 1/a$. Too small a step crawls; too large a step overshoots the bottom by more than it started and the error grows without limit. Figure 2 (right) shows exactly that for $\eta = 0.09$ on our data: the loss explodes. For the two-parameter bowl, the steepest direction of our loss has curvature about 23.7, so the stability limit is $\eta < 2/23.7 \approx 0.085$.

The slow crawl happens because the bowl is a long narrow valley: steep across, shallow along. Subtracting the mean from x before training makes the bowl rounder, and gradient descent converges far faster. Centring and rescaling inputs is standard practice in neural-network training for the same reason

8. Many inputs at once: the matrix form

With d inputs, writing every sum out is tedious. A neat trick removes the special treatment of the bias: add a fake input $x_0 = 1$ to every example and call its weight $w_0 = b$. Then $\hat{y} = \sum_{j=0}^{d} w_j x_j$ with no separate b. Stack the examples into a table X with n rows (one per example) and $d + 1$ columns (first column all ones), the targets into a column y, and the weights into a column w. All n predictions are then one matrix product:

$$\hat{y} = X\,w$$

The loss becomes the squared length of the error column, divided by n:

$$L(w) = \frac{1}{n}(y - Xw)^T(y - Xw)$$

Differentiating (the matrix version of the chain rule) gives the whole gradient in one line, which contains our two earlier formulas as special cases:

$$\nabla_w L = -\frac{2}{n}\,X^T(y - Xw)$$

Setting it to zero gives the **normal equations** and, when X^TX can be inverted, the closed-form solution:

$$X^TX\,w = X^Ty \quad \Longrightarrow \quad w = (X^TX)^{-1}X^Ty$$

On our data X has rows $(1, x_i)$, and the normal equations are just two linear equations in the unknowns b and w:

$$5\,b + 15\,w = 18.7, \quad 15\,b + 55\,w = 63.6$$

Eliminating b gives $w = (5 \cdot 63.6 - 15 \cdot 18.7)/(5 \cdot 55 - 15 \cdot 15) = 37.5/50 = 0.75$, and then $b = 1.49$, the same answer for the third time. Gradient descent in this notation is a single line, and it is the template for training every model in this series:

$$w \leftarrow w + \frac{2\eta}{n}\,X^T(y - Xw)$$

Why bother with gradient descent when a formula exists? Inverting X^TX costs time growing like d^3, fails when two inputs are perfectly correlated, and works for *only* this model. The descent loop works for any differentiable model, including networks.

9. Why squares? A probabilistic view

Choosing squared error can be justified from an assumption about how the data came to be. Suppose the true relationship is a line, but every measurement is jostled by independent random noise ε_i following a bell curve (a Gaussian) with mean 0 and variance σ^2:

$$y_i = w\,x_i + b + \varepsilon_i, \quad \varepsilon_i \sim N(0, \sigma^2)$$

Then, for given (w, b), the probability density of seeing the value y_i is

$$p(y_i|x_i) = \frac{1}{\sqrt{2\pi\sigma^2}}\, exp - \frac{(y_i - \hat{y}_i)^2}{2\sigma^2}$$

A sensible principle is to pick the parameters that make the observed data *most probable*: maximum likelihood. Multiplying the densities of independent examples and taking the logarithm (which turns products into sums and does not change where the maximum is) gives the log-likelihood:

$$\ell = -\frac{n}{2} ln(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

The first term does not involve w or b. So maximising ℓ is *exactly* the same as minimising the sum of squared errors. Least squares is therefore not an arbitrary choice: it is the right loss *if the noise is Gaussian*.

10. Assumptions and pitfalls

- **Linearity.** The method fits straight lines. If the true pattern curves, the residuals will show a systematic bend. Plot them.
- **Outliers.** Squaring makes a single wild point very influential. Exercise 1 shows one changed mark moving the slope by more than 50%.
- **Extrapolation.** The line is trustworthy only inside the range of x it has seen. Predicting for 20 hours of study is a guess about a world the data never visited.
- **The intercept.** b is the prediction at $x = 0$, which may be meaningless (a car with zero weight, a student with zero hours). Treat it as a position for the line, not a finding.
- **Constant spread and independence.** The Gaussian argument assumed noise of similar size everywhere and independent between examples. Violations make the fit less reliable.
- **Correlation is not causation.** The slope describes association in the data. It does not say what would happen if we *made* someone study longer.
- **Too many inputs.** With enough weights a model can memorise noise (overfitting). Controlling this is a central theme of later parts.

11. From a line to a neuron: the bridge to neural networks

Look again at the model $\hat{y} = w_1x_1 + \cdots + w_dx_d + b$. Draw each input as a dot, each weight as an arrow carrying that input to a single output node, and add the bias: this is precisely one

artificial neuron whose output is left as it is (an "identity" activation). Linear regression *is* a one-neuron network, trained with squared-error loss by gradient descent.

In linear regression	In a neural network
Inputs $x_1, \ldots, x_d$	The input layer
Weights w_j and bias b	The parameters that training adjusts
$\hat{y} = w \cdot x + b$	The neuron's weighted sum, usually called z
No extra transformation	An activation function (ReLU, sigmoid, …) applied to z
Mean squared error	A loss chosen to match the task (squared error, cross-entropy, …)
Walking downhill with $\partial L/\partial w$	Gradient descent and its variants
Partial derivatives computed by hand	Backpropagation: the chain rule applied automatically across many layers

The chain rule is already backpropagation

For a single example with loss $L_i = (y_i - \hat{y}_i)^2$, the derivative with respect to the weight splits into two easy pieces, "how does the loss change with the prediction" times "how does the prediction change with the weight":

$$\frac{\partial L_i}{\partial w} = \frac{\partial L_i}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial w} = -2(y_i - \hat{y}_i) \cdot x_i$$

Averaging over examples reproduces the gradient from Section 7. In a deep network the same idea is applied repeatedly, layer by layer, like a relay passing the error backwards. That is backpropagation, and it will not feel strange when we reach it.

Why a network needs more than lines

Suppose we stack two such units, feeding the output of the first into the second:

$$\hat{y} = w_2\,(w_1 x + b_1) + b_2 = (w_2 w_1)\,x + (w_2 b_1 + b_2)$$

The result is *another straight line*, with slope $w_2 w_1$ and intercept $w_2 b_1 + b_2$. Stacking linear layers adds no expressive power, however many we pile up. The remedy is to bend the output of each neuron with a nonlinear **activation function** before passing it on. That single change is what lets networks fit curves, images and language, and it is the subject of Part 2.

Appendix: notation

Symbol	Meaning
x, x_i	Input (feature); the input of example i
y, y_i	Target; the target of example i
$\hat{y}$	The model's prediction
n, d	Number of examples; number of input features
w, b	Weight (slope) and bias (intercept): the parameters
$\bar{x}, \bar{y}$	Means of the inputs and targets
e_i	Residual $y_i - \hat{y}_i$
L	Loss; here the mean squared error
SSE, SST	Sum of squared errors; total sum of squares
R^2	Share of variation explained by the model
$\partial L/\partial w$	Partial derivative: how L changes when only w is nudged
∇L	Gradient: the list of all partial derivatives
η	Learning rate
X, w, y	Data matrix, weight column, target column (Section 8)